



binary serialization and deserialization

Version 1.0.0

Contents

1	Introduction	11
1.1	Motivation	11
1.2	History	11
1.3	Primary I/O classes	11
1.4	Directly supported types	11
1.5	Supported std class templates	11
1.6	Concepts	11
2	Design	12
2.1	A primary I/O class for each source/sink type	12
2.2	Conceptual interfaces	12
2.3	Categories of interface overloads in actual implementation	13
2.3.1	Regular functions for directly supported types	13
2.3.2	Function templates for STL containers and other standard class templates	13
2.3.3	Function templates for types that meet the conceptual requirements of a given primary I/O class	14
2.4	Dwm::StreamIO read/write functions	14
2.4.1	Dwm::StreamIO conceptual read/write functions	14
2.5	Dwm::FileIO read/write functions	15
2.5.1	Dwm::FileIO conceptual read/write functions	15
2.6	Dwm::DescriptorIO read/write functions	15
2.6.1	Dwm::DescriptorIO conceptual read/write functions	15
2.7	Dwm::BZ2IO read/write functions	15
2.7.1	Dwm::BZ2IO conceptual read/write functions	15
2.8	Dwm::GZIO read/write functions	16
2.8.1	Dwm::GZIO conceptual read/write functions	16
2.9	Member function overloads	16
2.10	Member function templates	16

<i>CONTENTS</i>	3
2.11 Concepts	16
2.12 Support for user-designed types	16
2.12.1 Before C++26	16
2.12.2 C++26 and beyond	16
3 Examples	17
3.1 Write and read a string to/from an iostream	17
3.2 Write and read a map<string,string> to/from an iostream	18
3.3 Turtles all the way down	19
3.4 Integrating a user-defined type	20
3.4.1 With C++26 reflection	20
3.4.2 Without C++26 reflection	20
3.4.3 A test program to write and read std::vector<MyType>	21
3.4.4 Skipping data when using C++26 reflection	22
4 Primary I/O Classes	23
4.1 Dwm::StreamIO	24
4.1.1 Using Dwm::StreamIO with user defined types	24
4.2 Dwm::FileIO	25
4.2.1 Using Dwm::FileIO with user defined types	25
4.3 Dwm::DescriptorIO	26
4.3.1 Using Dwm::DescriptorIO with user defined types	26
4.4 Dwm::BZ2IO	27
4.4.1 Using Dwm::BZ2IO with user defined types	27
4.5 Dwm::GZIO	28
4.5.1 Using Dwm::GZIO with user defined types	28
4.6 Dwm::ASIO	28
5 Other I/O classes, functions and function templates	29
5.1 Dwm::SizedLength	29
5.2 Dwm::IOUtils	29

5.3	Endian conversions	29
5.3.1	template <typename T> concept IsEndianSensitiveInteger	29
5.3.2	template <typename T> requires IsEndianSensitiveInteger<T> inline auto BE2Host(T t) → T	29
5.3.3	template <typename T> requires IsEndianSensitiveInteger<T> inline auto Host2BE(T t) → T	30
5.3.4	template <typename ...Args> requires (IsEndianSensitiveInteger<Args> and ...) void BE2Host(Args & ...args)	30
5.3.5	template <typename ...Args> requires (IsEndianSensitiveInteger<Args> and ...) void Host2BE(Args & ...args)	30
6	Directly supported data types	31
6.1	integral types	31
6.2	floating point types	31
6.3	enumerated types	31
6.4	std::string	31
6.5	std::vector<bool>	32
7	Supported std type templates	33
7.1	std::array<class T, std::size_t N>	33
7.2	std::atomic<class T>	33
7.3	std::deque<class T>	33
7.4	std::list<class T>	34
7.5	std::map<class KeyType, class MappedType>	34
7.6	std::multimap<class KeyType, class MappedType>	34
7.7	std::optional<class T>	35
7.8	std::pair<class FirstType, class SecondType>	35
7.9	std::set<class T>	35
7.10	std::multiset<class T>	35
7.11	std::unique_ptr<class T>	36
7.12	std::unordered_map<class KeyType, class MappedType>	36

7.13	<code>std::unordered_multimap<class KeyType, class MappedType></code>	36
7.14	<code>std::unordered_set<class T></code>	36
7.15	<code>std::unordered_multiset<class T></code>	37
7.16	<code>std::vector<class T></code>	37
8	Concepts	38
8.1	Streamable	38
8.2	Concepts used by <code>Dwm::StreamIO</code> (C++ iostreams)	38
8.2.1	<code>HasStreamRead<T></code>	38
8.2.2	<code>HasStreamWrite<T></code>	38
8.2.3	<code>Dwm::IsStreamReadable<T></code>	38
8.2.4	<code>Dwm::IsStreamWritable<T></code>	38
8.3	Concepts used by <code>Dwm::FileIO</code> (FILES, i.e. <code>cstdio</code>)	39
8.3.1	<code>HasFileRead<T></code>	39
8.3.2	<code>HasFileWrite<T></code>	39
8.3.3	<code>Dwm::IsFileReadable<T></code>	39
8.3.4	<code>Dwm::IsFileWritable<T></code>	39
8.4	Concepts used by <code>Dwm::DescriptorIO</code> (UNIX descriptors)	39
8.4.1	<code>HasDescriptorRead<T></code>	39
8.4.2	<code>HasDescriptorWrite<T></code>	39
8.4.3	<code>Dwm::IsDescriptorReadable<T></code>	39
8.4.4	<code>Dwm::IsDescriptorWritable<T></code>	39
8.5	Boost asio sockets	40
8.5.1	<code>HasAsioRead<T></code>	40
8.5.2	<code>HasAsioWrite<T></code>	40
8.6	Concepts used by <code>Dwm::BZ2IO</code> (BZFILES from <code>BZ2_bzopen()</code>)	40
8.6.1	<code>HasBZRead<T></code>	40
8.6.2	<code>HasBZWrite<T></code>	40
8.6.3	<code>Dwm::IsBZ2Readable<T></code>	40
8.6.4	<code>Dwm::IsBZ2Writable<T></code>	40

8.7 Concepts used by <code>Dwm::GZIO</code> (<code>gzFiles</code> from <code>gzopen()</code>)	40
8.7.1 <code>HasGZRead<T></code>	40
8.7.2 <code>HasGZWrite<T></code>	40
8.7.3 <code>Dwm::IsGZReadable<T></code>	40
8.7.4 <code>Dwm::IsGZWritable<T></code>	40
8.8 <code>HasSkipAnnotation<std::meta::info info></code>	40
8.9 Streamed length	41
8.9.1 Constant streamed length	41
8.9.2 Constant streamed length via static <code>constexpr uint64_t StreamedLength()</code> member.	41
8.9.3 Constant streamed length from C++26 reflection	41
9 On-the-wire formats	42
9.1 On-the-wire formats, network byte order (MSB first)	42
9.1.1 16 bit integral types (<code>int16_t</code> , <code>uint16_t</code>)	42
9.1.2 32 bit integral types (<code>int32_t</code> , <code>uint32_t</code>)	42
9.1.3 64 bit integral types (<code>int64_t</code> , <code>uint64_t</code>)	42
9.1.4 <code>float</code>	42
9.1.5 <code>double</code>	42
9.1.6 <code>Dwm::SizedLength</code> encoding.	43
9.1.7 <code>std::string</code> encoding	46
9.1.8 <code>std::optional<T></code> encoding.	46
9.1.9 <code>std::pair<class FirstType, class SecondType></code> encoding	46
9.1.10 <code>std::vector<class T></code> encoding	46
9.1.11 <code>std::deque<T></code> encoding	47
9.1.12 <code>std::list<T></code> encoding	47
9.1.13 <code>std::map<class KeyType, class ValueType></code> encoding	47
9.1.14 <code>std::multimap<class KeyType, class ValueType></code> encoding.	48
9.1.15 <code>std::unordered_map<class KeyType, class ValueType></code> encoding	48
9.1.16 <code>std::unordered_multimap<class KeyType, class ValueType></code> encoding	48
9.1.17 <code>std::set<class T></code> encoding	48
9.1.18 <code>std::multiset<class T></code> encoding	48

<i>CONTENTS</i>	7
-----------------	---

9.1.19 <code>std::unordered_set<class T> encoding</code>	49
--	----

9.1.20 <code>std::unordered_multiset<class T> encoding</code>	49
---	----

List of Figures

9.1	Dwm::SizedLength encoding	43
9.2	Encoding byte	43
9.3	One byte value 0xAA, big-endian.	43
9.4	One byte value 0xAA, little-endian.	43
9.5	Two byte value 0xAABB, big-endian.	44
9.6	Two byte value 0xAABB, little-endian.	44
9.7	Three byte value 0xAABCC, big-endian.	44
9.8	Three byte value 0xAABCC, little-endian.	44
9.9	Four byte value 0xAABCCDD, big-endian.	44
9.10	Four byte value 0xAABCCDD, little-endian.	44
9.11	Five byte value 0xAABCCDDEE, big-endian.	44
9.12	Five byte value 0xAABCCDDEE, little-endian.	45
9.13	Six byte value 0xAABCCDDEEFF, big-endian.	45
9.14	Six byte value 0xAABCCDDEEFF, little-endian.	45
9.15	Seven byte value 0xAABCCDDEEFF99, big-endian.	45
9.16	Seven byte value 0xAABCCDDEEFF99, little-endian.	45
9.17	Eight byte value 0xAABCCDDEEFF9988, big-endian.	45
9.18	Eight byte value 0xAABCCDDEEFF9988, little-endian.	46
9.19	std::string encoding	46
9.20	std::optional<T> encoding	46
9.21	std::pair<class FirstType, class SecondType> encoding	46
9.22	std::vector<T> encoding	47
9.23	std::deque<T> encoding	47
9.24	std::list<T> encoding	47
9.25	std::map<KeyType,ValueType> encoding	47
9.26	std::multimap<KeyType,ValueType> encoding	48

9.27 <code>std::unordered_map<KeyType,ValueType></code> encoding	48
9.28 <code>std::unordered_multimap<Key,Value></code> encoding	48
9.29 <code>std::set<T></code> encoding	48
9.30 <code>std::multiset<T></code> encoding	48
9.31 <code>std::unordered_set<T></code> encoding	49
9.32 <code>std::unordered_multiset<T></code> encoding	49

List of Tables

2.1	Primary I/O class source and sink types	12
2.2	Conceptual read/write functions in all primary I/O classes	12
2.3	Conceptual interface mapping to the primary I/O classes	13
2.4	Primary I/O class concepts for user-defined types	14
2.5	Conceptual read and write member functions in <code>Dwm::StreamIO</code>	14

Chapter 1

Introduction

1.1 Motivation

Efficiency I did not want conversions between textual and native representations for arithmetic types. For example, if I need to stream an unsigned 32-bit integer of value 0xFFFFFFFF, I should only need to transfer 4 bytes, not 10 or more.

Portability Since network transport was a primary goal, I was motivated to provide schemes that work across fairly common hardware platforms and UNIX operating systems.

Flexibility I/O facilities should be flexible enough to accomodate instances of common class templates from the standard library, particularly containers like `vector`, `map`, et. al. They should also be flexible enough to accomodate user-defined types without a tremendous amount of work.

1.2 History

libDwm was started around 1998. Needless to say, C++ was a very different language in 1998 versus 2025. And the standard library has grown tremendously since 1998.

The I/O facilities in libDwm started appearing in the early 2000's (around the time we got C++03). Since that time, it has evolved a bit with each new version of the C++ standard, and is about to see a sea change with C++26 reflection (P2996 and P3394 in particular).

1.3 Primary I/O classes

1.4 Directly supported types

1.5 Supported std class templates

1.6 Concepts

Chapter 2

Design

2.1 A primary I/O class for each source/sink type

The library divides I/O functionality into a primary I/O class for a given type of source and sink. For example, `Dwm::StreamIO` provides functions for `std::istream` sources and `std::ostream` sinks. Table 2.1 lists the primary I/O classes and their associated source and sink type.

Primary I/O class	Source type	Sink type	Description
<code>Dwm::StreamIO</code>	<code>std::istream &</code>	<code>std::ostream &</code>	references to C++ iostreams
<code>Dwm::FileIO</code>	<code>FILE *</code>	<code>FILE *</code>	FILE created with <code>fopen()</code> and friends
<code>Dwm::DescriptorIO</code>	<code>int</code>	<code>int</code>	UNIX descriptors
<code>Dwm::BZ2IO</code>	<code>BZFILE *</code>	<code>BZFILE *</code>	BZFILE created with <code>BZ2_bzopen()</code>
<code>Dwm::GZIO</code>	<code>gzFile</code>	<code>gzFile</code>	gzFile created with <code>gzopen()</code>

Table 2.1: Primary I/O class source and sink types

Each of the primary I/O classes provides a set of static functions and function templates to read and write data.

2.2 Conceptual interfaces

Conceptually, the I/O classes provide four primary interfaces to read and write from/to their associated source/sink type, as depicted in table 2.2. There are single-object interfaces (`ReadFunc` and `WriteFunc`) and multi-object variadic interfaces (`VariadicReadFunc` and `VariadicWriteFunc`). The multi-object variadic interfaces just call to the single-object interfaces for each object.

Read/write functions (conceptual)
<pre>template <typename T> static ReadReturnType & ReadFunc(SourceType source, T & t); template <typename T> static WriteReturnType WriteFunc(SinkType sink, const T & t); template <typename... Args> static ReadReturnType VariadicReadFunc(SourceType source, Args & ...args); template <typename... Args> static WriteReturnType VariadicWriteFunc(SinkType sink, const Args & ...args);</pre>

Table 2.2: Conceptual read/write functions in all primary I/O classes

Table 2.3 maps the generic conceptual interface to each of the primary I/O classes.

	Dwm I/O class				
	StreamIO	FileIO	DescriptorIO	BZ2IO	GZIO
<i>ReadFunc</i>	Read	Read	Read	BZRead	Read
<i>VariadicReadFunc</i>	ReadV	ReadV	ReadV	BZReadV	ReadV
<i>ReadReturnType</i>	std::istream &	size_t	ssize_t	int	int
<i>SourceType</i>	std::istream &	FILE *	int	BZFILE *	gzFile
<i>WriteFunc</i>	Write	Write	Write	BZWrite	Write
<i>VariadicWriteFunc</i>	WriteV	WriteV	WriteV	BZWriteV	WriteV
<i>WriteReturnType</i>	std::ostream &	size_t	ssize_t	int	int
<i>SinkType</i>	std::ostream &	FILE *	int	BZFILE *	gzFile

Table 2.3: Conceptual interface mapping to the primary I/O classes

2.3 Categories of interface overloads in actual implementation

There are three categories of overloads in the implementation:

- Regular functions for directly supported types
- Function templates for STL containers and other standard class templates
- Function templates for types that meet the conceptual requirements of a given primary I/ class

2.3.1 Regular functions for directly supported types

Reading and writing of directly supported types is done with regular function overloads. For example, this function from `Dwm::StreamIO` to write a `uint32_t` value:

```
static std::ostream & Write(std::ostream & os, uint32_t value);
```

2.3.2 Function templates for STL containers and other standard class templates

Overloaded function templates are used to read and write instantiations of standard class templates, where the parameter types of the standard class templates are either directly supported types, types that meet the conceptual requirements of a readable and writable class, or instances of supported standard templates using any of the aforementioned types in type parameters. In other words, supported standard containers of any depth can be read and written, as long as the leaf types are directly supported types or types that meet the conceptual requirements of a readable and writable class for the I/O class of interest.

This is probably best explained via some example readable and writable types:

```
std::vector<std::string>
std::map<uint32_t, std::vector<std::string>>
std::map<uint32_t, std::vector<std::map<uint8_t, std::string>>>
```

An example of one such function template from `Dwm::StreamIO`:

```
template <typename valueT, size_t N>
static std::istream & Read(std::istream & is, std::array<valueT, N> & a)
```

2.3.3 Function templates for types that meet the conceptual requirements of a given primary I/O class

Overloaded function templates with concept requirements are used to read and write types that meet the concept requirements. The concepts required for each primary I/O class are listed in table 2.4.

An example of one such function template from `Dwm::StreamIO`:

```
static std::ostream & Write(std::ostream & os, const HasStreamWrite auto & t)
```

Dwm I/O class	Read concept	Write concept
<code>StreamIO</code>	<code>Dwm::HasStreamRead<T></code>	<code>Dwm::HasStreamWrite<T></code>
<code>FileIO</code>	<code>Dwm::HasFileRead<T></code>	<code>Dwm::HasFileWrite<T></code>
<code>DescriptorIO</code>	<code>Dwm::HasDescriptorRead<T></code>	<code>Dwm::HasDescriptorWrite<T></code>
<code>BZ2IO</code>	<code>Dwm::HasBZRead<T></code>	<code>Dwm::HasBZWrite<T></code>
<code>GZIO</code>	<code>Dwm::HasGZRead<T></code>	<code>Dwm::HasGZWrite<T></code>

Table 2.4: Primary I/O class concepts for user-defined types

2.4 `Dwm::StreamIO` read/write functions

2.4.1 `Dwm::StreamIO` conceptual read/write functions

Table 2.5 lists the conceptual read and write member functions in `Dwm::StreamIO`.

Primary I/O class	Read/write functions (conceptual)
<code>Dwm::StreamIO</code>	<code>template <typename T></code>
	<code>static std::istream & Read(std::istream & is, T & t);</code>
	<code>template <typename T></code>
	<code>static std::ostream & Write(std::ostream & os, const T & t);</code>
	<code>template <typename... Args></code>
	<code>static std::istream & ReadV(std::istream & is, Args & ...args);</code>
	<code>template <typename... Args></code>
	<code>static std::ostream & WriteV(std::ostream & os, const Args & ...args);</code>

Table 2.5: Conceptual read and write member functions in `Dwm::StreamIO`

Conceptually...

```
template <typename T>
static std::istream & Read(std::istream & is, T & t)
```

- Reads the contents of `t` from `is` and returns `is`.

```
template <typename T>
static std::ostream & Write(std::ostream & os, const T & t)
```

- Writes the contents of `t` to `os` and returns `os`.

```
template <typename... Args>
static std::istream & ReadV(std::istream & is, Args & ...args)
```

- Reads the contents of all `args` (in order) from `is` and returns `is`.

```
template <typename... Args>
static std::ostream & WriteV(std::ostream & os, const Args & ...args)
```

- Writes the contents of all `args` (in order) to `os` and returns `os`.

2.5 Dwm::FileIO read/write functions

2.5.1 Dwm::FileIO conceptual read/write functions

Primary I/O class	Read/write functions (conceptual)
Dwm::FileIO	template <typename T>
	static size_t Read(FILE *f, T & t);
	template <typename T>
	static size_t Write(FILE *f, const T & t);
	template <typename... Args>
	static size_t ReadV(FILE *f, Args & ...args);
	template <typename... Args>
	static size_t WriteV(FILE *f, const Args & ...args);

2.6 Dwm::DescriptorIO read/write functions

2.6.1 Dwm::DescriptorIO conceptual read/write functions

Primary I/O class	Read/write functions (conceptual)
Dwm::DescriptorIO	template <typename T>
	static ssize_t Read(int fd, T & t);
	template <typename T>
	static ssize_t Write(int fd, const T & t);
	template <typename... Args>
	static ssize_t ReadV(int fd, Args & ...args);
	template <typename... Args>
	static ssize_t WriteV(int fd, const Args & ...args);

2.7 Dwm::BZ2IO read/write functions

2.7.1 Dwm::BZ2IO conceptual read/write functions

Primary I/O class	Read/write functions (conceptual)
Dwm::BZ2IO	template <typename T>
	static int BZRead(BZFILE *bzf, T & t);
	template <typename T>
	static int BZWrite(BZFILE *bzf, const T & t)
	template <typename... Args>
	static int BZReadV(BZFILE *bzf, Args & ...args);
	template <typename T>
	static int BZWriteV(BZFILE *bzf, const Args & ...args)

2.8 Dwm::GZIO read/write functions

2.8.1 Dwm::GZIO conceptual read/write functions

Primary I/O class	Read/write functions (conceptual)
Dwm::GZIO	template <typename T>
	static int Read(gzFile gzf, T & t);
	template <typename T>
	static int Write(gzFile gzf, const T & t);
	template <typename... Args>
	static int ReadV(gzFile gzf, Args & ...args);
	template <typename... Args>
	static int Write(gzFile gzf, const Args & ...args);

2.9 Member function overloads

libDwm uses member function overloading to allow the user to use the same function names to read or write different types from/to a source/sink. This is also critical to supporting instances of class templates.

2.10 Member function templates

libDwm uses function templates and function template overloads to provide the same interfaces (the same member function names) to read or write instances of standard class templates from/to a source/sink (`std::vector`, `std::pair`, `std::tuple`, `std::map`, et. al.).

2.11 Concepts

libDwm uses concepts in many places to aid in overload resolution and to document the requirements of user-defined types.

2.12 Support for user-designed types

2.12.1 Before C++26

In general, code utilized with pre-C++26 compilers and standard libraries requires a pair of member functions to be added to any user-defined type for a given source/sink pair. Much of the time, these functions are fairly simple to implement, particularly for C++ iostreams (`std::istream` as a source, `std::ostream` as a sink). This is especially true for user-defined types that only contain types already supported by the corresponding primary I/O class (for C++ iostreams, that would be `Dwm::StreamIO`).

2.12.2 C++26 and beyond

With C++26 features (particularly P2996 and P3394), it becomes possible to elegantly handle serialization and deserialization of many user defined types with no changes to user code. It also allows annotations of types and variables that can control the serialization and deserialization of a user defined type (for example, skipping a data member of a class during serialization and deserialization). Work here has already begun, using a slightly modified fork of the bloomberg clang implementation.

Chapter 3

Examples

3.1 Write and read a string to/from an iostream

The program below writes a `std::string` to a `std::stringstream`, then reads a `std::string` from the `std::stringstream`. The program in listing [3.1](#) returns 0 if the string we read matches the string we wrote.

```
1 #include <sstream>
2 #include "DwmStreamIO.hh"
3
4 int main(int argc, char *argv[])
5 {
6     using std::string, std::stringstream;
7     bool      success{false};
8     string     writeString("Hello, world!"), readString;
9     stringstream ss;
10    if (Dwm::StreamIO::Write(ss, writeString)) {
11        if (Dwm::StreamIO::Read(ss, readString)) {
12            success = (readString == writeString);
13        }
14    }
15    return success ? 0 : 1;
16 }
```

Listing 3.1: Write and read a string to/from an iostream

3.2 Write and read a `map<string,string>` to/from an `iostream`

The program in listing 3.2 writes a `std::map<std::string,std::string>` to a `std::stringstream`, then reads a `std::map<std::string,std::string>` from the `std::stringstream` and returns 0 if the map we read matches the map we wrote. This is an example of our support for standard containers holding supported types (`std::string` is an explicitly supported type).

```
1 #include <sstream>
2 #include "DwmStreamIO.hh"
3
4 int main(int argc, char *argv[])
5 {
6     using stringmap = std::map<std::string,std::string>;
7     stringmap m1 {
8         { "apple",    "fruit"    },
9         { "cat",      "feline"   },
10        { "broccoli", "vegetable" },
11        { "dog",       "canine"   },
12        { "kiwi",      "fruit"    },
13        { "cow",       "bovine"   }
14    };
15    bool success{false};
16    std::stringstream ss;
17    if (Dwm::StreamIO::Write(ss, m1)) {
18        stringmap m2;
19        if (Dwm::StreamIO::Read(ss, m2)) {
20            success = (m2 == m1);
21        }
22    }
23    return success ? 0 : 1;
24 }
```

Listing 3.2: Write and read a `map<string,string>` to/from an `iostream`

3.3 Turtles all the way down

The program in listing 3.3 writes a `std::map<std::string, std::vector<std::string>>` to a `std::stringstream`, then reads a `std::map<std::string, std::vector<std::string>>` from the `std::stringstream` and returns 0 if the map we read matches the map we wrote. This is an example of our support for standard containers and supported types being “turtles all the way down”: our support is recursive to any depth.

```
1 #include <sstream>
2 #include "DwmStreamIO.hh"
3
4 int main(int argc, char *argv[])
5 {
6     using container = std::map<std::string, std::vector<std::string>>;
7     container m1 {
8         { "fruit",      { "apple", "banana", "kiwi", "orange" } },
9         { "canine",     { "husky", "labrador", "poodle", "wolf", "collie" } },
10        { "feline",      { "cougar", "lion", "panther" } },
11        { "vegetable", { "broccoli", "carrot", "squash" } }
12    };
13
14    bool success{false};
15    std::stringstream ss;
16    if (Dwm::StreamIO::Write(ss, m1)) {
17        container m2;
18        if (Dwm::StreamIO::Read(ss, m2)) {
19            success = (m2 == m1);
20        }
21    }
22    return success ? 0 : 1;
23 }
```

Listing 3.3: Write and read a `map<string, vector<string>>` to/from an `iostream`

3.4 Integrating a user-defined type

Listing 3.4 shows a user defined type `MyType` with three simple data members: `id`, `name` and `category`.

```

1 #include <cstdint>
2 #include <string>
3
4 struct MyType {
5     uint32_t    id;
6     std::string name;
7     std::string category;
8 };

```

Listing 3.4: `MyType` without explicit `Dwm::StreamIO` support

3.4.1 With C++26 reflection

With C++26 reflection, all of the [primary I/O classes](#) can read and write instances of `MyType` from/to their associated source/sink with no changes to `MyType`. The primary I/O classes will do what one would expect: read and write each of the members of `MyType`, in the order in which they appear in the declaration of `MyType`.

3.4.2 Without C++26 reflection

Without C++26 reflection, two members need to be added to `MyType` for each [primary I/O class](#) you wish to use with `MyType`. For example, listing 3.5 adds support for `Dwm::StreamIO` by adding the `Read(std::istream &)` and `Write(std::ostream &) const` member functions for the case where C++26 reflection is not available (`DWM_CAN_USE_REFLECTION` is not defined). Note that these member functions are simple in this case because our member data types are directly supported by `Dwm::StreamIO` and hence we can use the `ReadV()` and `WriteV()` members of `Dwm::StreamIO` to implement these members.

```

1 #include "DwmStreamIO.hh"
2
3 struct MyType {
4     uint32_t    id;
5     std::string name;
6     std::string category;
7
8     #if ! defined(DWM_CAN_USE_REFLECTION)
9         std::istream & Read(std::istream & is)
10        { return Dwm::StreamIO::ReadV(is, id, name, category); }
11
12        std::ostream & Write(std::ostream & os) const
13        { return Dwm::StreamIO::WriteV(os, id, name, category); }
14    #endif
15 };

```

Listing 3.5: `MyType` with explicit `Dwm::StreamIO` support when needed

3.4.3 A test program to write and read `std::vector<MyType>`

The program in listing 3.6 utilizes a user-defined type `MyType` that has explicit `Dwm::StreamIO` functionality as needed. `MyType` meets the requirements of the `HasStreamRead<T>` and `HasStreamWrite<T>` concepts by providing the `Read(std::istream &)` member at lines 10-11 and the `Write(std::ostream &) const` member at lines 13-14 when we do not have C++26 reflection.

```

1  #include <sstream>
2  #include "DwmStreamIO.hh"
3
4  struct MyType {
5      uint32_t    id;
6      std::string name;
7      std::string category;
8
9  #if ! defined(DWM_CAN_USE_REFLECTION)
10     std::istream & Read(std::istream & is)
11     { return Dwm::StreamIO::ReadV(is, id, name, category); }
12
13     std::ostream & Write(std::ostream & os) const
14     { return Dwm::StreamIO::WriteV(os, id, name, category); }
15 #endif
16
17     bool operator <=> (const MyType &) const = default; // default comparison
18 };
19
20 int main(int argc, char *argv[])
21 {
22     bool                rc{false};
23     std::vector<MyType> myvec1 {
24         {1, "cat", "feline"},
25         {2, "dog", "canine" },
26         {3, "cow", "bovine" }
27     };
28     std::stringstream    ss;
29     if (Dwm::StreamIO::Write(ss, myvec1)) {
30         std::vector<MyType> myvec2;
31         if (Dwm::StreamIO::Read(ss, myvec2)) {
32             rc = (myvec2 == myvec1);
33         }
34     }
35     return rc ? 0 : 1;
36 }

```

Listing 3.6: Integrating a user-defined type

The main function creates a vector of `MyType` named `myvec1` on line 23. It writes `myvec1` to the `stringstream` named `ss` at line 29. It then creates a second vector of `MyType` named `myvec2` on line 30 and reads the contents of `myvec2` from `ss` on line 31. On line 32 it checks if the vector we read (`myvec2`) matches the vector we wrote (`myvec1`).

3.4.4 Skipping data when using C++26 reflection

Chapter 4

Primary I/O Classes

There are several primary I/O classes in libDwm. Each is used with a particular type of source and sink.

`Dwm::StreamIO` is used with C++ iostreams (`std::istream` and `std::ostream`).

`Dwm::FileIO` is used with `FILE`s created via `fopen()`, `fdopen()`, et. al.

`Dwm::DescriptorIO` is used with UNIX file descriptors created via `open()`, et. al.

`Dwm::BZ2IO` is used with bz2 streams (BZFILES opened via `BZ2_bzopen()`).

`Dwm::GZIO` is used with gz streams (gzFiles opened via `gzopen()`).

When adding I/O support for your own data type (struct or class), `Dwm::StreamIO` support would typically be the first target. That's largely because for fixed length types, all of the other I/O classes can be supported via an intermediate pass through `Dwm::StreamIO` with a `std::stringstream` as a data buffer.

4.1 Dwm::StreamIO

source type: `std::istream`

sink type: `std::ostream`

`Dwm::StreamIO` provides static functions and function templates to read and write from/to C++ iostreams. Conceptually, the primary interface looks like four static member function templates:

```
namespace Dwm {
    class StreamIO {
    public:
        template <typename T>
        static std::istream & Read(std::istream & is, T & t);

        template <typename T>
        static std::ostream & Write(std::ostream & os, const T & t);

        template <typename... Args>
        static std::istream & ReadV(std::istream & is, Args & ...args);

        template <typename... Args>
        static std::ostream & WriteV(std::ostream & os, const Args & ...args);
    };
}
```

The actual implementation uses many function and function template overloads for `Read()` and `Write()`. It can't automatically read and write any type `T`; a data type must either be directly supported or meet the requirements of `Dwm::HasStreamRead` and `Dwm::HasStreamWrite`.

Note that `ReadV()` and `WriteV()` are just convenient variadic function templates that will read or write multiple objects in sequence.

4.1.1 Using `Dwm::StreamIO` with user defined types

If a user-defined type provides the following two public members, `Dwm::StreamIO` will utilize them to read and write instances of the user defined type:

```
std::istream & Read(std::istream & is)
std::ostream & Write(std::ostream & os) const
```

See the `Dwm::HasStreamRead` and `Dwm::HasStreamWrite` concepts for the semantic requirements of these members.

4.2 Dwm::FileIO

source type: FILE *

sink type: FILE *

Dwm::FileIO provides static functions and function templates to read and write from/to FILES. Conceptually, the interface looks like four static member function templates:

```
namespace Dwm {
    class FileIO {
    public:
        template <typename T>
        static size_t Read(FILE *f, T & t);

        template <typename T>
        static size_t Write(FILE *f, const T & t);

        template <typename... Args>
        static size_t ReadV(FILE *f, Args & ...args);

        template <typename... Args>
        static size_t WriteV(FILE *f, const Args & ...args);
    };
}
```

The actual implementation uses many function and function template overloads for Read() and Write(). It can't automatically read and write any type T; a data type must either be directly supported or meet the requirements of [Dwm::HasFileRead](#) and [Dwm::HasFileWrite](#).

Note that ReadV() and WriteV() are just convenient variadic function templates that will read or write multiple objects in sequence.

4.2.1 Using Dwm::FileIO with user defined types

If a user-defined type provides the following two public members, Dwm::FileIO will utilize them to read and write instances of the user defined type:

```
size_t Read(FILE *f)
size_t Write(FILE *f) const
```

See the [Dwm::HasFileRead](#) and [Dwm::HasFileWrite](#) concepts for the semantic requirements of these members.

4.3 Dwm::DescriptorIO

source type: int
sink type: int

Dwm::DescriptorIO provides static functions and function templates to read and write from/to UNIX file descriptors. Conceptually, the interface looks like four static member function templates:

```
namespace Dwm {
    class DescriptorIO {
    public:
        <template typename T> static ssize_t Read(int fd, T & t);

        <template typename T> static ssize_t Write(int fd, const T & t);

        template <typename ...Args>
        static ssize_t ReadV(int fd, Args & ...args);

        template <typename ...Args>
        static ssize_t WriteV(int fd, const Args & ...args);
    };
}
```

The actual implementation uses many function and function template overloads for Read() and Write(). It can't automatically read and write any type T; a data type must either be directly supported or meet the requirements of Dwm::HasDescriptorRead and Dwm::HasDescriptorWrite.

Note that ReadV() and WriteV() are just convenient variadic function templates that will read or write multiple objects in sequence.

4.3.1 Using Dwm::DescriptorIO with user defined types

If a user defined type provides the following two public members, Dwm::DescriptorIO will utilize them to read and write instances of the user defined type:

```
ssize_t Read(int fd)
ssize_t Write(int fd) const
```

See the Dwm::HasDescriptorRead and Dwm::HasDescriptorWrite concepts for the semantic requirements of these members.

4.4 Dwm::BZ2IO

source type: BZFILE *

sink type: BZFILE *

Dwm::BZ2IO provides static functions and function templates to read and write from/to bz2 streams (BZFILE *). Conceptually, the interface looks like four static member function templates:

```
namespace Dwm {
    class BZ2IO {
    public:
        <template typename T>
        static int BZRead(BZFILE *bzf, T & t);

        <template typename T>
        static int BZWrite(BZFILE *bzf, const T & t);

        template <typename ...Args>
        static int BZReadV(BZFILE *bzf, Args & ...args);

        template <typename ...Args>
        static int BZWriteV(BZFILE *bzf, const Args & ...args);
    };
}
```

The actual implementation uses many function and function template overloads for BZRead() and BZWrite(). It can't automatically read and write any type T; a data type must either be directly supported or meet the requirements of [Dwm::HasBZRead](#) and [Dwm::HasBZWrite](#).

4.4.1 Using Dwm::BZ2IO with user defined types

If a user defined type provides the following two public members, Dwm::BZ2IO will utilize them to read and write instances of the user defined type:

```
int BZRead(BZFILE *)
int BZWrite(BZFILE *) const
```

See the [Dwm::HasBZRead](#) and [Dwm::HasBZWrite](#) concepts for the semantic requirements of these member functions.

4.5 Dwm::GZIO

gzFile source type: gzFile
 sink type: gzFile

Dwm::GZIO provides static functions and function templates to read and write from/to gz streams (gzFile). Conceptually, the interface looks like four static member function templates:

```
namespace Dwm {
    class GZIO {
    public:
        <template typename T>
        static int Read(gzFile gzf, T & t);

        <template typename T>
        static int Write(gzFile gzf, const T & t);

        template <typename ...Args>
        static int ReadV(gzFile gzf, Args & ...args);

        template <typename ...Args>
        static int WriteV(gzFile gzf, const Args & ...args);
    };
}
```

The actual implementation uses many function and function template overloads for Read() and Write(). It can't automatically read and write any type T; a data type must either be directly supported or meet the requirements of [Dwm::HasGZRead](#) and [Dwm::HasGZWrite](#).

4.5.1 Using Dwm::GZIO with user defined types

If a user defined type provides the following two public members, Dwm::GZIO will utilize them to read and write instances of the user defined type:

```
int Read(gzFile)
int Write(gzFile) const
```

See the [Dwm::HasGZRead](#) and [Dwm::HasGZWrite](#) concepts for the expected semantics of these members.

4.6 Dwm::ASIO

Boost asio sockets

Chapter 5

Other I/O classes, functions and function templates

5.1 `Dwm::SizedLength`

`Dwm::SizedLength` represents a `uint64_t` value, but on the wire is encoded as an encoding byte followed by a variable length (1 to 8) sequence of bytes containing the value. It is a typical space/time tradeoff: it requires more time to read or write, but saves space on the wire and storage for some common use cases.

Common use cases generally revolve around `std::size_t` values representing the length of a `std::string`, the number of elements in an STL container (vector, map, et. al.), or the return of the `sizeof` operator. On a 64-bit platform, you may need to accomodate 64-bit length values, but find that all of your values are less than 2^{32} . Or less than 256. These are scenarios where `Dwm::SizedLength` may be useful if the space/time tradeoff is desired.

See [Dwm::SizedLength on-the-wire encoding](#) for details on the encoding format.

5.2 `Dwm::IOUtils`

5.3 Endian conversions

5.3.1 `template <typename T>` `concept IsEndianSensitiveInteger`

True if `std::is_integral_v<T>` and `sizeof(T)` is 2, 4 or 8.

Examples:

```
IsEndianSensitiveInteger<uint64_t> → true
IsEndianSensitiveInteger<uint32_t> → true
IsEndianSensitiveInteger<uint16_t> → true
IsEndianSensitiveInteger<uint8_t>  → false
IsEndianSensitiveInteger<char>     → false
IsEndianSensitiveInteger<float>    → false
```

5.3.2 `template <typename T>` `requires IsEndianSensitiveInteger<T>` `inline auto BE2Host(T t) → T`

Returns a `T` holding the value of `t` converted from big endian (MSB first) byte order to host byte order. Example:

```
uint16_t x = 0xFF00;
uint16_t y = Dwm::BE2Host(x);
// on a little endian machine, y == 0x00FF
assert(0x00FF == y);
```

**5.3.3 template <typename T>
 requires IsEndianSensitiveInteger<T>
 inline auto Host2BE(T t) → T**

Returns a T holding the value of t converted from host byte order to big-endian (MSB first) byte order. Example:

```
uint32_t x = 0xAABBCCDD;
uint32_t y = Dwm::Host2BE(x);
// on a little endian machine, y == 0xDDCCBBAA
assert(0xDDCCBBAA == y);
```

**5.3.4 template <typename ...Args>
 requires (IsEndianSensitiveInteger<Args> and ...)
 void BE2Host(Args & ...args)**

**5.3.5 template <typename ...Args>
 requires (IsEndianSensitiveInteger<Args> and ...)
 void Host2BE(Args & ...args)**

Chapter 6

Directly supported data types

Directly supported data types are those types for which we have static member functions to read the type from a source and write the type to a sink. These types include integral types, floating point types, enumerated types, `std::string` and `std::vector<bool>`.

6.1 integral types

The category of types for which `std::is_integral_v<T>` evaluates to true.

Size	Supported type	typically aliased from
1 byte	<code>bool</code>	
	<code>char</code>	
	<code>int8_t</code>	<code>char</code>
	<code>uint8_t</code>	<code>unsigned char</code>
2 bytes	<code>int16_t</code>	<code>short</code>
	<code>uint16_t</code>	<code>unsigned short</code>
4 bytes	<code>int32_t</code>	<code>int</code> or <code>long</code>
	<code>uint32_t</code>	<code>unsigned int</code> or <code>unsigned long</code>
8 bytes	<code>int64_t</code>	<code>long long</code>
	<code>uint64_t</code>	<code>unsigned long long</code>

For multi-byte types in this category, we use an MSB-first representation on-the-wire.

6.2 floating point types

The category of types for which `std::is_floating_point_v<T>` evaluates to true.

Size	Supported type
4 bytes	<code>float</code>
8 bytes	<code>double</code>

We use XDR for the on-the-wire representation of `float` and `double`.

6.3 enumerated types

The category of types for which `std::is_enum_v<T>` evaluates to true. Note that in general, we expect an enumerated type to have an explicit underlying type so we are certain of its size.

6.4 `std::string`

`std::string` is explicitly supported.

6.5 `std::vector<bool>`

`std::vector<bool>` is explicitly supported, mainly because it's not a conforming container (which all of us in the C++ community wish was not true). In general it's unwise to use `std::vector<bool>` due to its lack of complete conformance with what's expected of an instance of `std::vector<T>`.

Chapter 7

Supported `std` type templates

7.1 `std::array<class T, std::size_t N>`

We can stream any instances of `std::array<T,N>` where `T` satisfies the read and write concepts of the desired primary I/O class.

primary I/O class	Read concept	Write concept
<code>Dwm::StreamIO</code>	<code>Dwm::IsStreamReadable<T></code>	<code>Dwm::IsStreamWritable<T></code>
<code>Dwm::FileIO</code>	<code>Dwm::IsFileReadable<T></code>	<code>Dwm::IsFileWritable<T></code>
<code>Dwm::DescriptorIO</code>	<code>Dwm::IsDescriptorReadable<T></code>	<code>Dwm::IsDescriptorWritable<T></code>
<code>Dwm::BZ2IO</code>	<code>Dwm::IsBZ2Readable<T></code>	<code>Dwm::IsBZ2Writable<T></code>
<code>Dwm::GZIO</code>	<code>Dwm::IsGZReadable<T></code>	<code>Dwm::IsGZWritable<T></code>

7.2 `std::atomic<class T>`

We can stream any instances of `std::atomic<T>` where `T` satisfies the read and write concepts of the desired primary I/O class. Note this excludes cases where `T` is a raw pointer, `std::shared_ptr` or `std::weak_ptr`.

primary I/O class	Read concept	Write concept
<code>Dwm::StreamIO</code>	<code>Dwm::IsStreamReadable<T></code>	<code>Dwm::IsStreamWritable<T></code>
<code>Dwm::FileIO</code>	<code>Dwm::IsFileReadable<T></code>	<code>Dwm::IsFileWritable<T></code>
<code>Dwm::DescriptorIO</code>	<code>Dwm::IsDescriptorReadable<T></code>	<code>Dwm::IsDescriptorWritable<T></code>
<code>Dwm::BZ2IO</code>	<code>Dwm::IsBZ2Readable<T></code>	<code>Dwm::IsBZ2Writable<T></code>
<code>Dwm::GZIO</code>	<code>Dwm::IsGZReadable<T></code>	<code>Dwm::IsGZWritable<T></code>

7.3 `std::deque<class T>`

We can stream any instances of `std::deque<T>` where `T` satisfies the read and write concepts of the desired primary I/O class.

primary I/O class	Read concept	Write concept
<code>Dwm::StreamIO</code>	<code>Dwm::IsStreamReadable<T></code>	<code>Dwm::IsStreamWritable<T></code>
<code>Dwm::FileIO</code>	<code>Dwm::IsFileReadable<T></code>	<code>Dwm::IsFileWritable<T></code>
<code>Dwm::DescriptorIO</code>	<code>Dwm::IsDescriptorReadable<T></code>	<code>Dwm::IsDescriptorWritable<T></code>
<code>Dwm::BZ2IO</code>	<code>Dwm::IsBZ2Readable<T></code>	<code>Dwm::IsBZ2Writable<T></code>
<code>Dwm::GZIO</code>	<code>Dwm::IsGZReadable<T></code>	<code>Dwm::IsGZWritable<T></code>

See `std::deque<class T> encoding` for encoding detail.

7.4 `std::list<class T>`

We can stream any instances of `std::list<T>` where `T` satisfies the read and write concepts of the desired primary I/O class.

primary I/O class	Read concept	Write concept
<code>Dwm::StreamIO</code>	<code>Dwm::IsStreamReadable<T></code>	<code>Dwm::IsStreamWritable<T></code>
<code>Dwm::FileIO</code>	<code>Dwm::IsFileReadable<T></code>	<code>Dwm::IsFileWritable<T></code>
<code>Dwm::DescriptorIO</code>	<code>Dwm::IsDescriptorReadable<T></code>	<code>Dwm::IsDescriptorWritable<T></code>
<code>Dwm::BZ2IO</code>	<code>Dwm::IsBZ2Readable<T></code>	<code>Dwm::IsBZ2Writable<T></code>
<code>Dwm::GZIO</code>	<code>Dwm::IsGZReadable<T></code>	<code>Dwm::IsGZWritable<T></code>

See `std::list<class T>` encoding for encoding detail.

7.5 `std::map<class KeyType, class MappedType>`

We can stream any instances of `std::map<KeyType, MappedType>` where `KeyType` and `MappedType` satisfy the read and write requirements of the desired primary I/O class.

primary I/O class	Read concept	Write concept
<code>Dwm::StreamIO</code>	<code>Dwm::IsStreamReadable<KeyType></code>	<code>Dwm::IsStreamWritable<KeyType></code>
	<code>Dwm::IsStreamReadable<MappedType></code>	<code>Dwm::IsStreamWritable<MappedType></code>
<code>Dwm::FileIO</code>	<code>Dwm::IsFileReadable<KeyType></code>	<code>Dwm::IsFileWritable<KeyType></code>
	<code>Dwm::IsFileReadable<MappedType></code>	<code>Dwm::IsFileWritable<MappedType></code>
<code>Dwm::DescriptorIO</code>	<code>Dwm::IsDescriptorReadable<KeyType></code>	<code>Dwm::IsDescriptorWritable<KeyType></code>
	<code>Dwm::IsDescriptorReadable<MappedType></code>	<code>Dwm::IsDescriptorWritable<MappedType></code>
<code>Dwm::BZ2IO</code>	<code>Dwm::IsBZ2Readable<KeyType></code>	<code>Dwm::IsBZ2Writable<KeyType></code>
	<code>Dwm::IsBZ2Readable<MappedType></code>	<code>Dwm::IsBZ2Writable<MappedType></code>
<code>Dwm::GZIO</code>	<code>Dwm::IsGZReadable<KeyType></code>	<code>Dwm::IsGZWritable<KeyType></code>
	<code>Dwm::IsGZReadable<MappedType></code>	<code>Dwm::IsGZWritable<MappedType></code>

See `std::map<class KeyType, class MappedType>` encoding for encoding detail.

7.6 `std::multimap<class KeyType, class MappedType>`

We can stream any instances of `std::multimap<KeyType, MappedType>` where `KeyType` and `MappedType` satisfy the read and write requirements of the desired primary I/O class.

primary I/O class	Read concept	Write concept
<code>Dwm::StreamIO</code>	<code>Dwm::IsStreamReadable<KeyType></code>	<code>Dwm::IsStreamWritable<KeyType></code>
	<code>Dwm::IsStreamReadable<MappedType></code>	<code>Dwm::IsStreamWritable<MappedType></code>
<code>Dwm::FileIO</code>	<code>Dwm::IsFileReadable<KeyType></code>	<code>Dwm::IsFileWritable<KeyType></code>
	<code>Dwm::IsFileReadable<MappedType></code>	<code>Dwm::IsFileWritable<MappedType></code>
<code>Dwm::DescriptorIO</code>	<code>Dwm::IsDescriptorReadable<KeyType></code>	<code>Dwm::IsDescriptorWritable<KeyType></code>
	<code>Dwm::IsDescriptorReadable<MappedType></code>	<code>Dwm::IsDescriptorWritable<MappedType></code>
<code>Dwm::BZ2IO</code>	<code>Dwm::IsBZ2Readable<KeyType></code>	<code>Dwm::IsBZ2Writable<KeyType></code>
	<code>Dwm::IsBZ2Readable<MappedType></code>	<code>Dwm::IsBZ2Writable<MappedType></code>
<code>Dwm::GZIO</code>	<code>Dwm::IsGZReadable<KeyType></code>	<code>Dwm::IsGZWritable<KeyType></code>
	<code>Dwm::IsGZReadable<MappedType></code>	<code>Dwm::IsGZWritable<MappedType></code>

See `std::multimap<class KeyType, class MappedType>` encoding for encoding detail.

7.7 `std::optional<class T>`

We can stream any instances of `std::optional<T>` where `T` satisfies the read and write concepts of the desired primary I/O class.

primary I/O class	Read concept	Write concept
<code>Dwm::StreamIO</code>	<code>Dwm::IsStreamReadable<T></code>	<code>Dwm::IsStreamWritable<T></code>
<code>Dwm::FileIO</code>	<code>Dwm::IsFileReadable<T></code>	<code>Dwm::IsFileWritable<T></code>
<code>Dwm::DescriptorIO</code>	<code>Dwm::IsDescriptorReadable<T></code>	<code>Dwm::IsDescriptorWritable<T></code>
<code>Dwm::BZ2IO</code>	<code>Dwm::IsBZ2Readable<T></code>	<code>Dwm::IsBZ2Writable<T></code>
<code>Dwm::GZIO</code>	<code>Dwm::IsGZReadable<T></code>	<code>Dwm::IsGZWritable<T></code>

See `std::optional<T>` [encoding](#) for encoding details.

7.8 `std::pair<class FirstType, class SecondType>`

We can stream any instances of `std::pair<FirstType, SecondType>` where `FirstType` and `SecondType` satisfy the read and write concepts of the desired primary I/O class.

primary I/O class	Read concept	Write concept
<code>Dwm::StreamIO</code>	<code>Dwm::IsStreamReadable<FirstType></code>	<code>Dwm::IsStreamWritable<FirstType></code>
	<code>Dwm::IsStreamReadable<SecondType></code>	<code>Dwm::IsStreamWritable<SecondType></code>
<code>Dwm::FileIO</code>	<code>Dwm::IsFileReadable<FirstType></code>	<code>Dwm::IsFileWritable<FirstType></code>
	<code>Dwm::IsFileReadable<SecondType></code>	<code>Dwm::IsFileWritable<SecondType></code>
<code>Dwm::DescriptorIO</code>	<code>Dwm::IsDescriptorReadable<FirstType></code>	<code>Dwm::IsDescriptorWritable<FirstType></code>
	<code>Dwm::IsDescriptorReadable<SecondType></code>	<code>Dwm::IsDescriptorWritable<SecondType></code>
<code>Dwm::BZ2IO</code>	<code>Dwm::IsBZ2Readable<FirstType></code>	<code>Dwm::IsBZ2Writable<FirstType></code>
	<code>Dwm::IsBZ2Readable<SecondType></code>	<code>Dwm::IsBZ2Writable<SecondType></code>
<code>Dwm::GZIO</code>	<code>Dwm::IsGZReadable<FirstType></code>	<code>Dwm::IsGZWritable<FirstType></code>
	<code>Dwm::IsGZReadable<SecondType></code>	<code>Dwm::IsGZWritable<SecondType></code>

See `std::pair<class FirstType, class FirstType>` [encoding](#) for encoding details.

7.9 `std::set<class T>`

We can stream any instances of `std::set<T>` where `T` satisfies the read and write concepts of the desired primary I/O class.

primary I/O class	Read concept	Write concept
<code>Dwm::StreamIO</code>	<code>Dwm::IsStreamReadable<T></code>	<code>Dwm::IsStreamWritable<T></code>
<code>Dwm::FileIO</code>	<code>Dwm::IsFileReadable<T></code>	<code>Dwm::IsFileWritable<T></code>
<code>Dwm::DescriptorIO</code>	<code>Dwm::IsDescriptorReadable<T></code>	<code>Dwm::IsDescriptorWritable<T></code>
<code>Dwm::BZ2IO</code>	<code>Dwm::IsBZ2Readable<T></code>	<code>Dwm::IsBZ2Writable<T></code>
<code>Dwm::GZIO</code>	<code>Dwm::IsGZReadable<T></code>	<code>Dwm::IsGZWritable<T></code>

See `std::set<class T>` [encoding](#) for encoding details.

7.10 `std::multiset<class T>`

We can stream any instances of `std::multiset<T>` where `T` satisfies the read and write concepts of the desired primary I/O class.

primary I/O class	Read concept	Write concept
<code>Dwm::StreamIO</code>	<code>Dwm::IsStreamReadable<T></code>	<code>Dwm::IsStreamWritable<T></code>
<code>Dwm::FileIO</code>	<code>Dwm::IsFileReadable<T></code>	<code>Dwm::IsFileWritable<T></code>
<code>Dwm::DescriptorIO</code>	<code>Dwm::IsDescriptorReadable<T></code>	<code>Dwm::IsDescriptorWritable<T></code>
<code>Dwm::BZ2IO</code>	<code>Dwm::IsBZ2Readable<T></code>	<code>Dwm::IsBZ2Writable<T></code>
<code>Dwm::GZIO</code>	<code>Dwm::IsGZReadable<T></code>	<code>Dwm::IsGZWritable<T></code>

See `std::multiset<class T>` [encoding](#) for encoding detail.

7.11 `std::unique_ptr<class T>`

We can stream instances of `std::unique_ptr<T>` where `T` is streamable and the `unique_ptr`'s `deleter_type` is `std::default_delete<T>`. We use the `deleter_type` test to infer that the `std::unique_ptr<T>` points to a single object and not to an array. This can of course be incorrect, and hence should not be relied on as a general mechanism.

7.12 `std::unordered_map<class KeyType, class MappedType>`

We can stream any instances of `std::unordered_map<KeyType, MappedType>` where `KeyType` and `MappedType` satisfy the read and write requirements of the desired primary I/O class.

primary I/O class	Read concept	Write concept
<code>Dwm::StreamIO</code>	<code>Dwm::IsStreamReadable<KeyType></code> <code>Dwm::IsStreamReadable<MappedType></code>	<code>Dwm::IsStreamWritable<KeyType></code> <code>Dwm::IsStreamWritable<MappedType></code>
<code>Dwm::FileIO</code>	<code>Dwm::IsFileReadable<KeyType></code> <code>Dwm::IsFileReadable<MappedType></code>	<code>Dwm::IsFileWritable<KeyType></code> <code>Dwm::IsFileWritable<MappedType></code>
<code>Dwm::DescriptorIO</code>	<code>Dwm::IsDescriptorReadable<KeyType></code> <code>Dwm::IsDescriptorReadable<MappedType></code>	<code>Dwm::IsDescriptorWritable<KeyType></code> <code>Dwm::IsDescriptorWritable<MappedType></code>
<code>Dwm::BZ2IO</code>	<code>Dwm::IsBZ2Readable<KeyType></code> <code>Dwm::IsBZ2Readable<MappedType></code>	<code>Dwm::IsBZ2Writable<KeyType></code> <code>Dwm::IsBZ2Writable<MappedType></code>
<code>Dwm::GZIO</code>	<code>Dwm::IsGZReadable<KeyType></code> <code>Dwm::IsGZReadable<MappedType></code>	<code>Dwm::IsGZWritable<KeyType></code> <code>Dwm::IsGZWritable<MappedType></code>

See `std::unordered_map<class KeyType, class MappedType>` encoding for encoding detail.

7.13 `std::unordered_multimap<class KeyType, class MappedType>`

We can stream any instances of `std::unordered_multimap<KeyType, MappedType>` where `KeyType` and `MappedType` satisfy the read and write requirements of the desired primary I/O class.

primary I/O class	Read concept	Write concept
<code>Dwm::StreamIO</code>	<code>Dwm::IsStreamReadable<KeyType></code> <code>Dwm::IsStreamReadable<MappedType></code>	<code>Dwm::IsStreamWritable<KeyType></code> <code>Dwm::IsStreamWritable<MappedType></code>
<code>Dwm::FileIO</code>	<code>Dwm::IsFileReadable<KeyType></code> <code>Dwm::IsFileReadable<MappedType></code>	<code>Dwm::IsFileWritable<KeyType></code> <code>Dwm::IsFileWritable<MappedType></code>
<code>Dwm::DescriptorIO</code>	<code>Dwm::IsDescriptorReadable<KeyType></code> <code>Dwm::IsDescriptorReadable<MappedType></code>	<code>Dwm::IsDescriptorWritable<KeyType></code> <code>Dwm::IsDescriptorWritable<MappedType></code>
<code>Dwm::BZ2IO</code>	<code>Dwm::IsBZ2Readable<KeyType></code> <code>Dwm::IsBZ2Readable<MappedType></code>	<code>Dwm::IsBZ2Writable<KeyType></code> <code>Dwm::IsBZ2Writable<MappedType></code>
<code>Dwm::GZIO</code>	<code>Dwm::IsGZReadable<KeyType></code> <code>Dwm::IsGZReadable<MappedType></code>	<code>Dwm::IsGZWritable<KeyType></code> <code>Dwm::IsGZWritable<MappedType></code>

See `std::unordered_multimap<class KeyType, class MappedType>` encoding for encoding detail.

7.14 `std::unordered_set<class T>`

We can stream any instances of `std::unordered_set<T>` where `T` satisfies the read and write concepts of the desired primary I/O class.

primary I/O class	Read concept	Write concept
<code>Dwm::StreamIO</code>	<code>Dwm::IsStreamReadable<T></code>	<code>Dwm::IsStreamWritable<T></code>
<code>Dwm::FileIO</code>	<code>Dwm::IsFileReadable<T></code>	<code>Dwm::IsFileWritable<T></code>
<code>Dwm::DescriptorIO</code>	<code>Dwm::IsDescriptorReadable<T></code>	<code>Dwm::IsDescriptorWritable<T></code>
<code>Dwm::BZ2IO</code>	<code>Dwm::IsBZ2Readable<T></code>	<code>Dwm::IsBZ2Writable<T></code>
<code>Dwm::GZIO</code>	<code>Dwm::IsGZReadable<T></code>	<code>Dwm::IsGZWritable<T></code>

See `std::unordered_set<class T>` encoding for encoding detail.

7.15 `std::unordered_multiset<class T>`

We can stream any instances of `std::unordered_multiset<T>` where `T` satisfies the read and write concepts of the desired primary I/O class.

primary I/O class	Read concept	Write concept
<code>Dwm::StreamIO</code>	<code>Dwm::IsStreamReadable<T></code>	<code>Dwm::IsStreamWritable<T></code>
<code>Dwm::FileIO</code>	<code>Dwm::IsFileReadable<T></code>	<code>Dwm::IsFileWritable<T></code>
<code>Dwm::DescriptorIO</code>	<code>Dwm::IsDescriptorReadable<T></code>	<code>Dwm::IsDescriptorWritable<T></code>
<code>Dwm::BZ2IO</code>	<code>Dwm::IsBZ2Readable<T></code>	<code>Dwm::IsBZ2Writable<T></code>
<code>Dwm::GZIOW</code>	<code>Dwm::IsGZReadable<T></code>	<code>Dwm::IsGZWritable<T></code>

See `std::unordered_multiset<class T>` [encoding](#) for encoding detail.

7.16 `std::vector<class T>`

We can stream any instances of `std::vector<T>` where `T` satisfies the read and write concepts of the desired primary I/O class.

primary I/O class	Read concept	Write concept
<code>Dwm::StreamIO</code>	<code>Dwm::IsStreamReadable<T></code>	<code>Dwm::IsStreamWritable<T></code>
<code>Dwm::FileIO</code>	<code>Dwm::IsFileReadable<T></code>	<code>Dwm::IsFileWritable<T></code>
<code>Dwm::DescriptorIO</code>	<code>Dwm::IsDescriptorReadable<T></code>	<code>Dwm::IsDescriptorWritable<T></code>
<code>Dwm::BZ2IO</code>	<code>Dwm::IsBZ2Readable<T></code>	<code>Dwm::IsBZ2Writable<T></code>
<code>Dwm::GZIOW</code>	<code>Dwm::IsGZReadable<T></code>	<code>Dwm::IsGZWritable<T></code>

See `std::vector<class T>` [encoding](#) for encoding detail.

Chapter 8

Concepts

8.1 Streamable

8.2 Concepts used by `Dwm::StreamIO` (C++ iostreams)

8.2.1 `HasStreamRead<T>`

Evaluates to true if T has a member function with signature:

```
std::istream & Read(std::istream & is)
```

Required semantics

- Must populate `*this` with contents read from `is`
- Must return `is`

Other semantics

- May set `std::ios_base::failbit`

8.2.2 `HasStreamWrite<T>`

Evaluates to true if T has a member function with signature:

```
std::ostream & Write(std::ostream & os) const
```

Required semantics

- Must write the contents of `*this` to `os`
- Must return `os`

Other semantics

- May set `std::ios_base::failbit`

8.2.3 `Dwm::IsStreamReadable<T>`

Evaluates to true if T is a directly supported type, a type that satisfies `HasStreamRead<T>`, an instance of a supported class template, or (if C++26 reflection is available) is readable via reflection.

8.2.4 `Dwm::IsStreamWritable<T>`

Evaluates to true if T is a directly supported type, a type that satisfies `HasStreamWrite<T>`, an instance of a supported class template, or (if C++26 reflection is available) is writable via reflection.

8.3 Concepts used by `Dwm::FileIO` (FILEs, i.e. `cstdio`)

8.3.1 `HasFileRead<T>`

Evaluates to true if T has a member function with signature:

```
size_t Read(FILE *f)
```

Required semantics

- Must populate `*this` with contents read from `f`
- Must return a non-zero `size_t` on success, 0 on failure

8.3.2 `HasFileWrite<T>`

Evaluates to true if T has a member function with signature:

```
size_t Write(FILE *f) const
```

Required semantics

- Must write the contents of `*this` to `f`
- Must return a non-zero `size_t` on success, 0 on failure

8.3.3 `Dwm::IsFileReadable<T>`

8.3.4 `Dwm::IsFileWritable<T>`

8.4 Concepts used by `Dwm::DescriptorIO` (UNIX descriptors)

8.4.1 `HasDescriptorRead<T>`

Evaluates to true if T has a member function with signature:

```
ssize_t Read(int fd)
```

Required semantics

- Must populate `*this` with contents read from `fd`
- Must return a positive `ssize_t` on success, -1 on failure

8.4.2 `HasDescriptorWrite<T>`

Evaluates to true if T has a member function with signature:

```
ssize_t Write(int fd) const
```

Required semantics

- Must write the contents of `*this` to `fd`
- Must return a positive `ssize_t` on success, -1 on failure

8.4.3 `Dwm::IsDescriptorReadable<T>`

8.4.4 `Dwm::IsDescriptorWritable<T>`

8.5 Boost asio sockets

8.5.1 HasAsioRead<T>

8.5.2 HasAsioWrite<T>

8.6 Concepts used by Dwm::BZ2IO (BZFILEs from BZ2_bzopen())

8.6.1 HasBZRead<T>

Evaluates to true if T has a member function with signature:

```
int BZRead(BZFILE *bzf)
```

Required semantics

- Must populate *this with contents read from bzf
- Must return a positive int on success, -1 on failure

8.6.2 HasBZWrite<T>

Evaluates to true if T has a member function with signature:

```
int Write(BZFILE *bzf) const
```

Required semantics

- Must write the contents of *this to bzf
- Must return a positive int on success, -1 on failure

8.6.3 Dwm::IsBZ2Readable<T>

8.6.4 Dwm::IsBZ2Writable<T>

8.7 Concepts used by Dwm::GZIO (gzFiles from gzopen())

8.7.1 HasGZRead<T>

Evaluates to true if T has a member function with signature:

```
int Read(gzFile gzf)
```

Required semantics

- Must populate *this with contents read from gzf
- Should return the number of decompressed bytes read on success, -1 on failure

8.7.2 HasGZWrite<T>

Evaluates to true if T has a member function with signature:

```
int Write(gzFile gzf) const
```

Required semantics

- Must write the contents of *this to gzf
- Should return the number of uncompressed bytes written on success, -1 on failure

8.7.3 Dwm::IsGZReadable<T>

8.7.4 Dwm::IsGZWritable<T>

8.8 HasSkipAnnotation<std::meta::info info>

Evaluates to true if the given reflection info has an annotation of the form [=Dwm::skip_io]

This requires C++26 reflection, specifically P2996 and P3394.

8.9 Streamed length

8.9.1 Constant streamed length

8.9.2 Constant streamed length via `static constexpr uint64_t StreamedLength()` member

8.9.3 Constant streamed length from C++26 reflection

When using C++26 reflection for serialization and deserialization of instances of a given type, we can determine at compile time whether or not the streamed length of the type is constant, and hence whether or not we can leverage `StreamIO` to provide `DescriptorIO`, `FileIO`, `BZ2IO`, `GZIO` and `ASIO` functionality automatically (via a pass through a `std::stringstream`).

Chapter 9

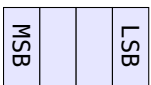
On-the-wire formats

9.1 On-the-wire formats, network byte order (MSB first)

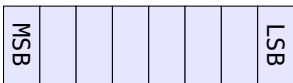
9.1.1 16 bit integral types (`int16_t`, `uint16_t`)



9.1.2 32 bit integral types (`int32_t`, `uint32_t`)

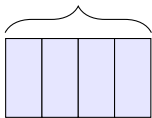


9.1.3 64 bit integral types (`int64_t`, `uint64_t`)



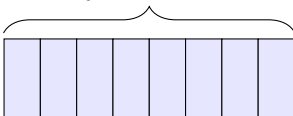
9.1.4 **float**

4 bytes, RFC4506



9.1.5 **double**

8 bytes, RFC4506



9.1.6 Dwm::SizedLength encoding

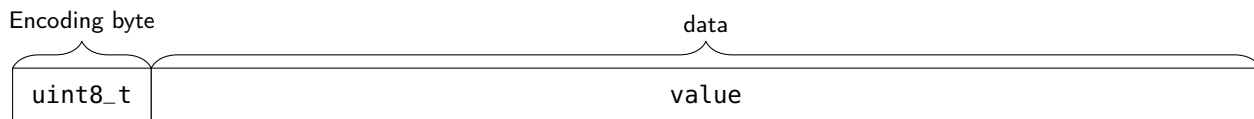


Figure 9.1: Dwm::SizedLength encoding

Encoding byte

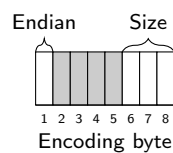


Figure 9.2: Encoding byte

The encoding byte uses 4 of the available 8 bits. The Endian bit indicates endianness (0 = little endian, 1 = big endian) of the value that follows the encoding byte. The Size bits indicate the length of the value (in bytes) following the encoding byte, minus 1:

$$\text{length of value} = \text{Size} + 1$$

The remaining bits in the encoding byte are unused.

The value of a Dwm::SizedLength is always written in native byte order (typically little-endian). The read member functions of Dwm::SizedLength will convert endianness if needed.

Examples

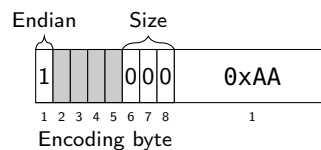


Figure 9.3: One byte value 0xAA, big-endian.

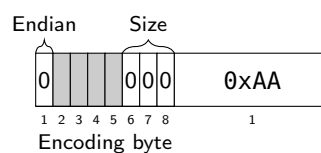


Figure 9.4: One byte value 0xAA, little-endian.

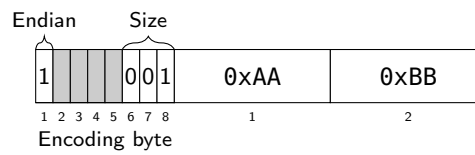


Figure 9.5: Two byte value 0xAABB, big-endian.

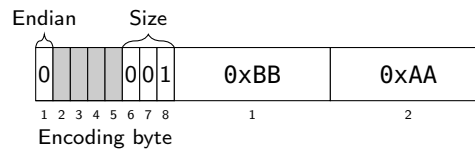


Figure 9.6: Two byte value 0xAABB, little-endian.

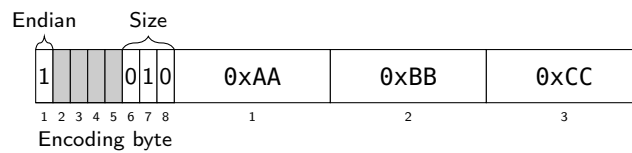


Figure 9.7: Three byte value 0xAABBCC, big-endian.

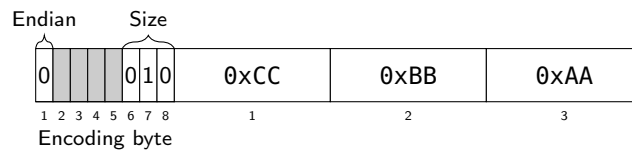


Figure 9.8: Three byte value 0xAABBCC, little-endian.



Figure 9.9: Four byte value 0xAABBCCDD, big-endian.

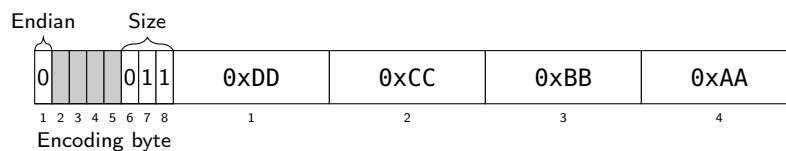


Figure 9.10: Four byte value 0xAABBCCDD, little-endian.



Figure 9.11: Five byte value 0xAABBCCDDEE, big-endian.

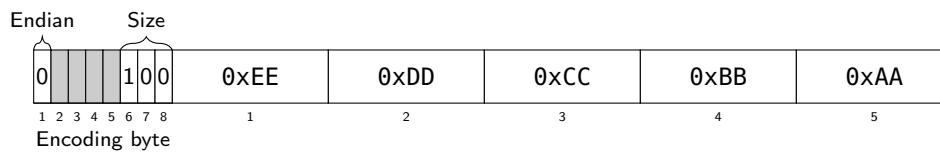


Figure 9.12: Five byte value 0xAABBCCDDEE, little-endian.

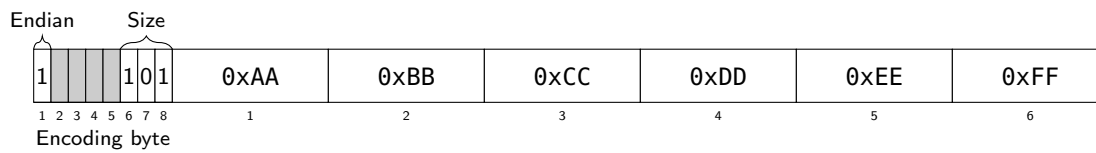


Figure 9.13: Six byte value 0xAABBCCDDEEFF, big-endian.



Figure 9.14: Six byte value 0xAABBCCDDEEFF, little-endian.

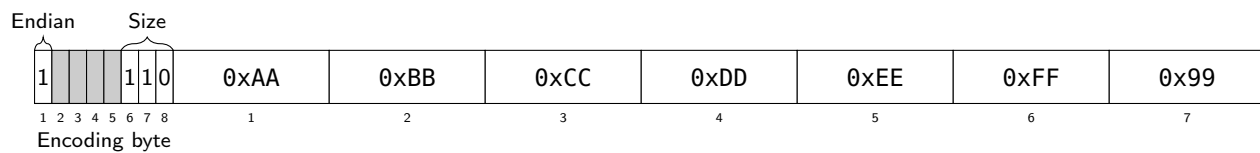


Figure 9.15: Seven byte value 0xAABBCCDDEEFF99, big-endian.

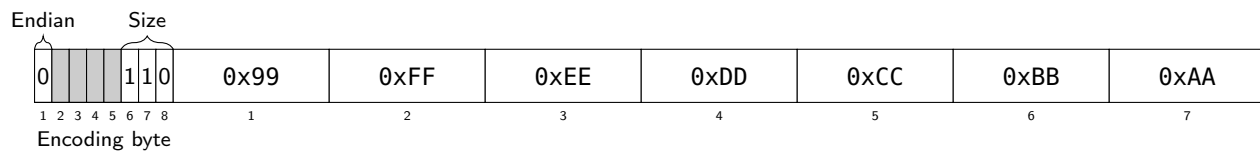


Figure 9.16: Seven byte value 0xAABBCCDDEEFF99, little-endian.

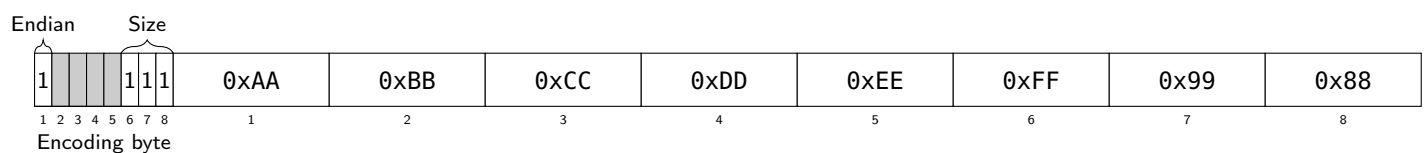


Figure 9.17: Eight byte value 0xAABBCCDDEEFF9988, big-endian.

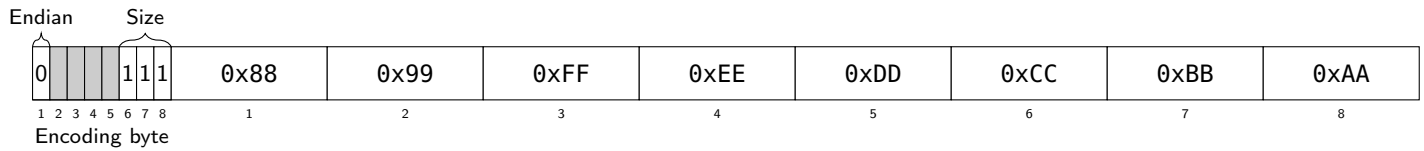
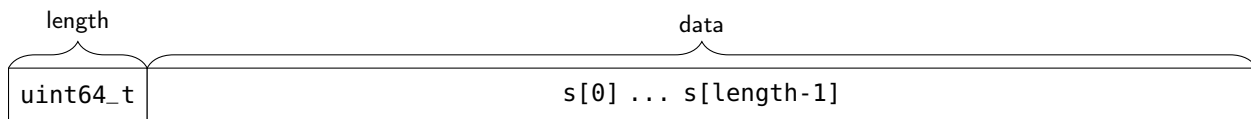


Figure 9.18: Eight byte value 0xAABBCCDDEEFF9988, little-endian.

9.1.7 `std::string` encoding

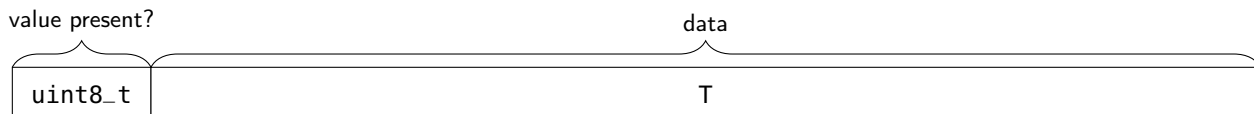
`std::string` is encoded with a 64-bit length field and then the characters in the string.

Figure 9.19: `std::string` encoding

NOTE: The length field is likely to be changed to use `Dwm::SizedLength` in the future.

9.1.8 `std::optional<T>` encoding

A `std::optional<T>` is encoded with a single byte to indicate whether or not the value is present, then the value if present.

Figure 9.20: `std::optional<T>` encoding

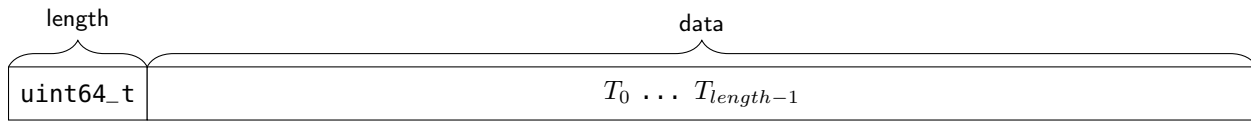
9.1.9 `std::pair<class FirstType, class SecondType>` encoding

The encoding of a `std::pair` is as you'd expect: the first value followed by the second value.

Figure 9.21: `std::pair<class FirstType, class SecondType>` encoding

9.1.10 `std::vector<class T>` encoding

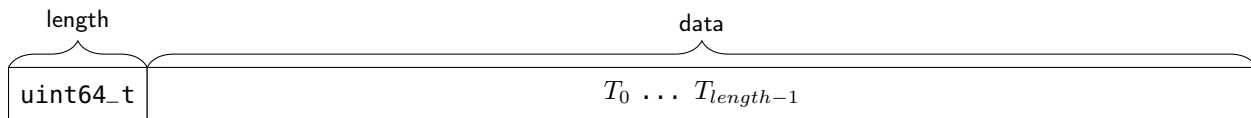
A `std::vector` is encoded with a 64-bit length field followed by length elements of the vector.

Figure 9.22: `std::vector<T>` encoding

NOTE: The length field is likely to be changed to use `Dwm::SizedLength` encoding in the future.

9.1.11 `std::deque<T>` encoding

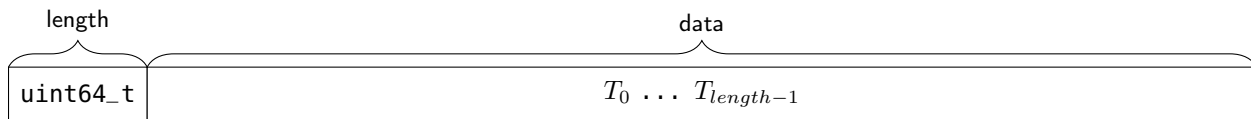
A `std::deque` is encoded with a 64-bit length field followed by `length` elements of the deque.

Figure 9.23: `std::deque<T>` encoding

NOTE: The length field is likely to be changed to use `Dwm::SizedLength` in the future.

9.1.12 `std::list<T>` encoding

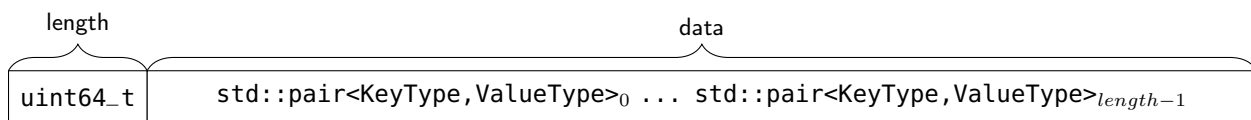
A `std::list` is encoded with a 64-bit length field followed by `length` elements of the list.

Figure 9.24: `std::list<T>` encoding

NOTE: The length field is likely to be changed to use `Dwm::SizedLength` in the future.

9.1.13 `std::map<class KeyType, class ValueType>` encoding

A `std::map` is encoded with a 64-bit length field followed by `length` pairs of (`KeyType`, `ValueType`) elements of the map.

Figure 9.25: `std::map<KeyType,ValueType>` encoding

NOTE: The length field is likely to be changed to use `Dwm::SizedLength` in the future.

9.1.14 `std::multimap<class KeyType, class ValueType>` encoding

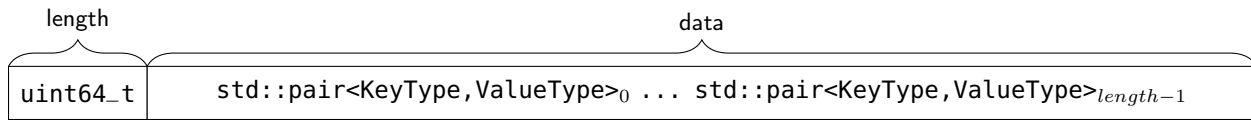


Figure 9.26: `std::multimap<KeyType,ValueType>` encoding

NOTE: The length field is likely to be changed to use `Dwm::SizedLength` in the future.

9.1.15 `std::unordered_map<class KeyType, class ValueType>` encoding

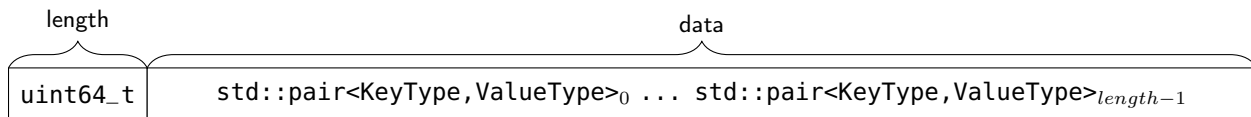


Figure 9.27: `std::unordered_map<KeyType,ValueType>` encoding

9.1.16 `std::unordered_multimap<class KeyType, class ValueType>` encoding

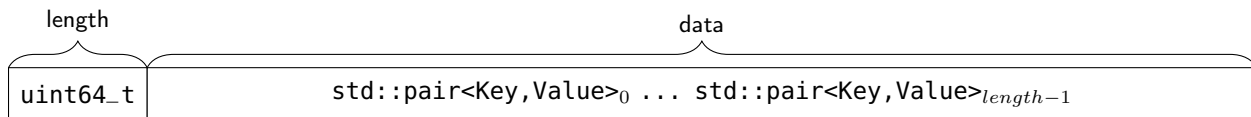


Figure 9.28: `std::unordered_multimap<Key,Value>` encoding

9.1.17 `std::set<class T>` encoding

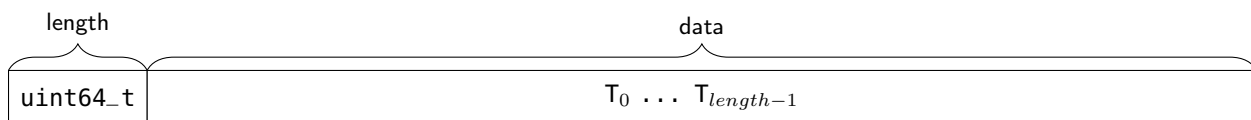


Figure 9.29: `std::set<T>` encoding

9.1.18 `std::multiset<class T>` encoding

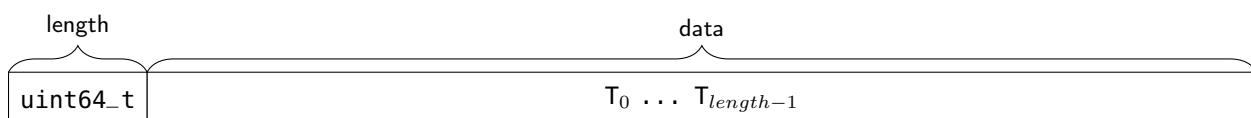
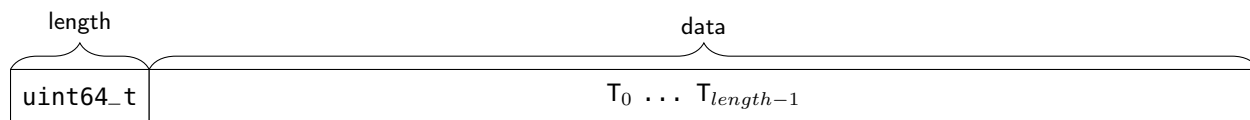
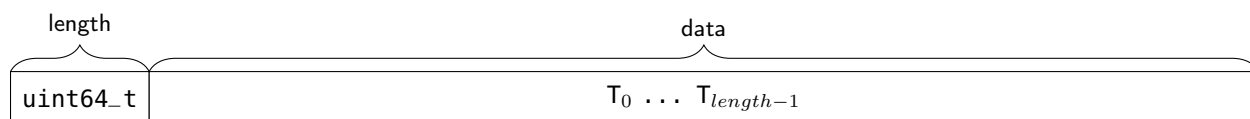


Figure 9.30: `std::multiset<T>` encoding

9.1.19 `std::unordered_set<class T>` encoding

Figure 9.31: `std::unordered_set<T>` encoding

9.1.20 `std::unordered_multiset<class T>` encoding

Figure 9.32: `std::unordered_multiset<T>` encoding